Correlation of Sunrise and Sunset Variations and Their Influence on Day Length

S. Naveera Fatima Rizvi

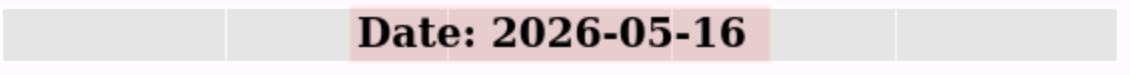
An exploratory analysis of sunrise and sunset timings from sunrise-and-sunset.com, focusing on their interrelationship and the associated variations in day length.

In [101…

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('ggplot')
%matplotlib inline
import matplotlib.dates as mdates
import seaborn as sns
import statsmodels.formula.api as smf
import datetime as dt

import plotly.io as pio
pio.renderers.default= 'notebook_connected'
import plotly.express as px
import plotly.graph_objects as go
```

In [102…

```
#This section of code display's today's date as a text in a matplotlib plot. Displaying ir

fig,ax= plt.subplots(figsize=(10,0.5))
ax.text(
    0.5,0.5,  #set position
    "Date: "+str(pd.to_datetime(dt.datetime.today().date()))[:11],
    bbox=dict(facecolor='red', alpha=0.1), #make a red colored box around the text
    fontsize=20,fontweight="bold",family='serif', #format
    horizontalalignment='center',
    verticalalignment='center',
    )
ax.tick_params(  #remove axes ticks and labels
    axis='both',
    bottom=False, left=False,
    labelbottom=False, labelleft=False)
fig.set_facecolor(tuple(np.array([255, 250, 255]) / 255)) #set facecolor for the figure
```



In [103…

```
# Read data
url = "https://www.sunrise-and-sunset.com/en/sun" #Set url
df= pd.read_html(url)[0] #Read the dataset


#Cleaning
df= df.droplevel(1,axis=1) #drop unnecessary column levels
df=df.query("Sunrise!='midnight sun' & Sunset!='midnight sun'" )#Drop midnightsun cities
df= df.drop_duplicates("Country",keep="first") #drop duplicates

#Convert to Sunrise, Sunset and Day length column values to the datetime format
for i in df.columns[2:]:
    df[i]= pd.to_datetime(df[i], format="%H:%M")
```

In [104…
```
#Confirming data types and checking for missing values
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 224 entries, 0 to 226
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype         
---  ------      --------------  -----         
 0   City        224 non-null    object        
 1   Country     224 non-null    object        
 2   Sunrise     224 non-null    datetime64[ns]
 3   Sunset      224 non-null    datetime64[ns]
 4   Day length  224 non-null    datetime64[ns]
dtypes: datetime64[ns](3), object(2)
memory usage: 10.5+ KB
```

Top 5 Cities with the Shortest Days

In [105…
```
def top_cities(metric:str):
    """
    Sorts the dataset by day length to extract temporal objects from the 10 most extreme r
    (either the shortest or longest days).

    Input: Metric: Shortest (for top 5 rows) or Longest (for bottom 5 rows)
    Output: Sorted dataframe for the required mettic
    """
    if metric == "shortest":
       disp_tab = df.sort_values("Day length").head(5)
    else:
       disp_tab = df.sort_values("Day length").tail(5)

    for i in disp_tab.columns[2:]:
        disp_tab[i]= disp_tab[i].dt.time
    return disp_tab.reset_index(drop=True)
```

In [106…
```
top_cities("shortest")
```

Out[106…

	City	Country	Sunrise	Sunset	Day length
0	Stanley	Falkland Islands	08:26:00	17:08:00	08:41:00
1	Wellington	New Zealand	07:23:00	17:10:00	09:46:00
2	Canberra	Australia	06:52:00	17:06:00	10:13:00
3	Montevideo	Uruguay	07:33:00	17:48:00	10:14:00
4	Buenos Aires	Argentina	07:41:00	17:57:00	10:15:00

Top 5 Cities with the Longest Days

In [107…
```
top_cities("longest")
```

Out[107…

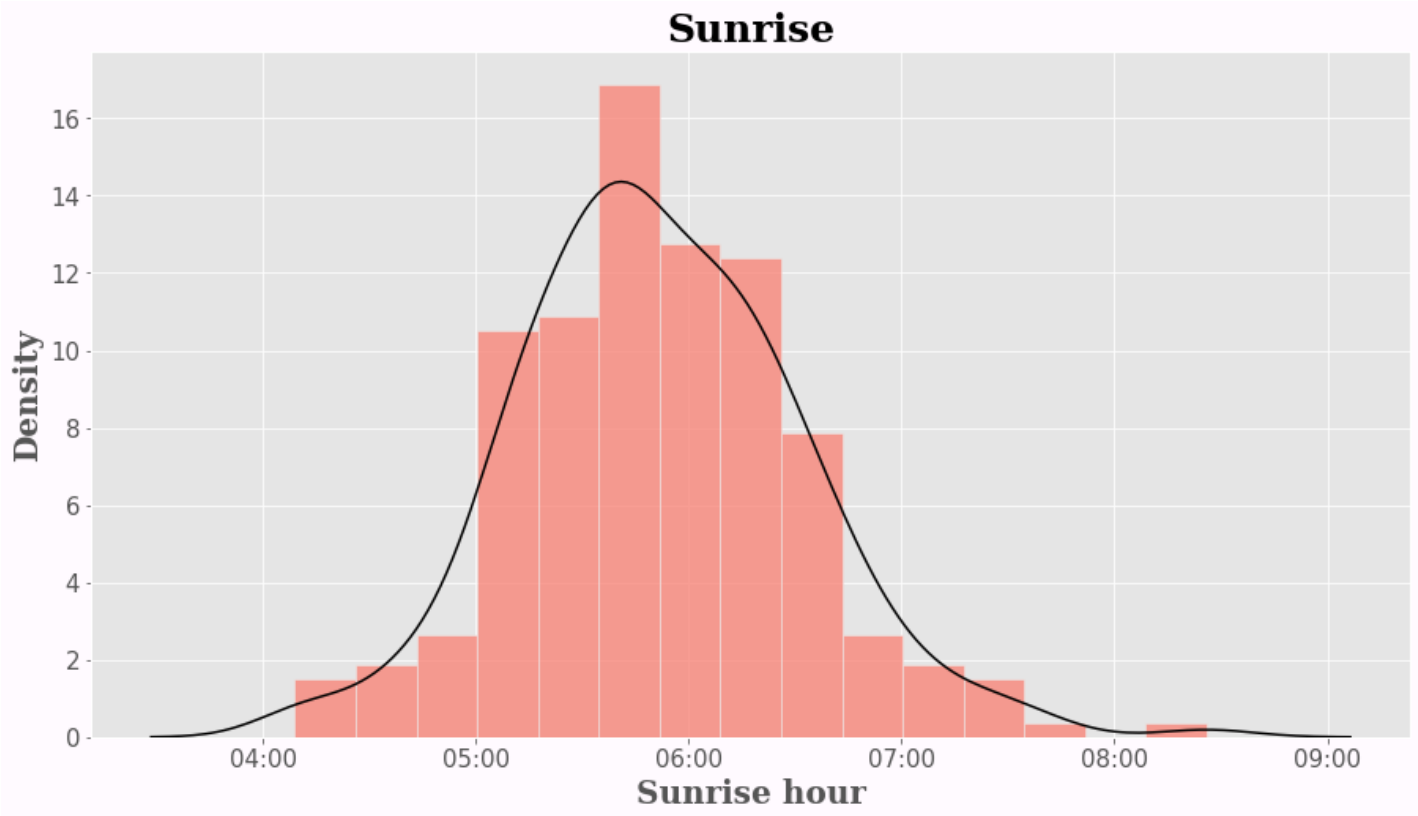
	City	Country	Sunrise	Sunset	Day length
0	Mariehamn	Åland	05:01:00	22:13:00	17:12:00
1	Helsinki	Finland	04:40:00	21:54:00	17:13:00
2	Tórshavn	Faroe Islands	04:32:00	22:16:00	17:44:00
3	Reykjavík	Iceland	04:11:00	22:39:00	18:27:00

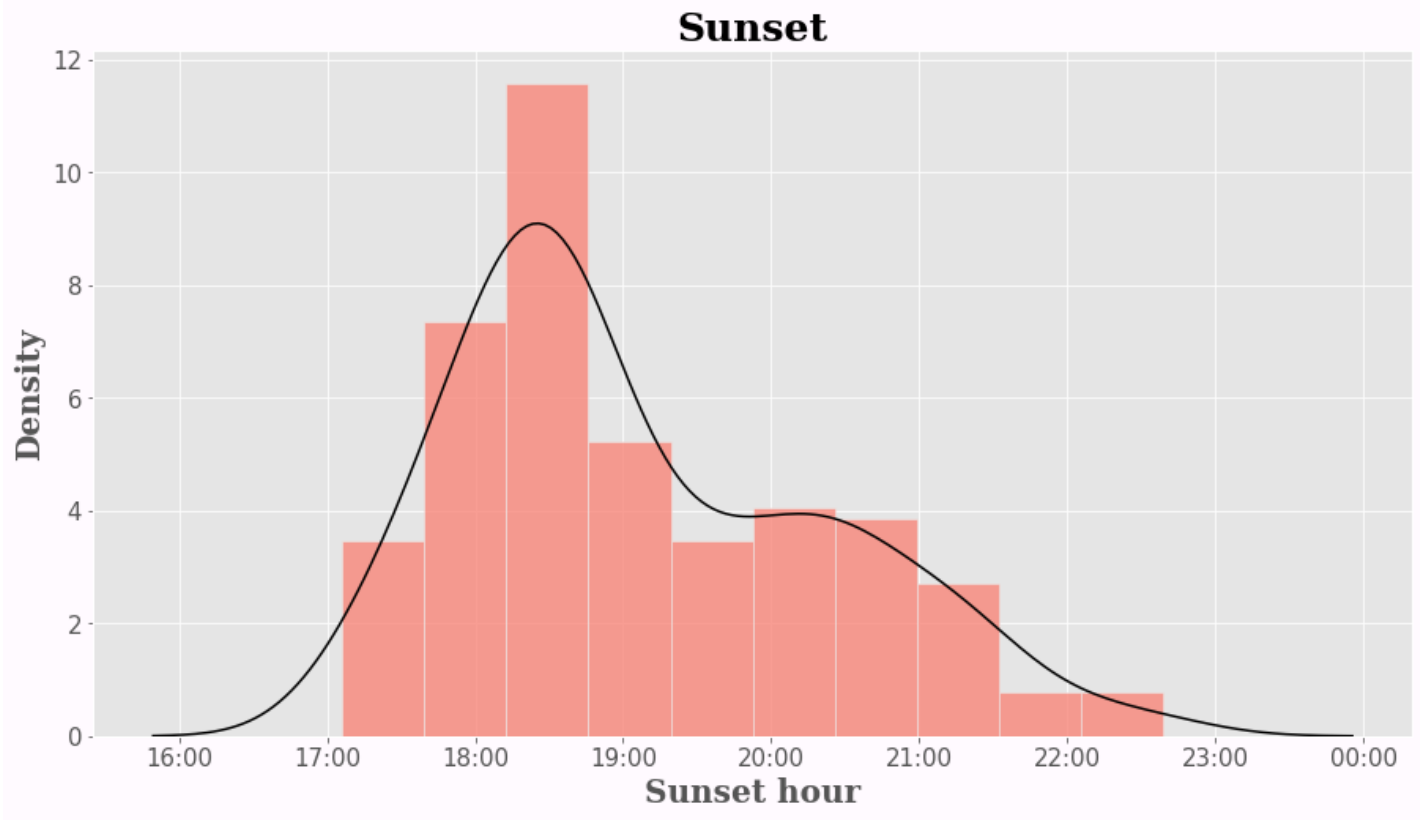
	City	Country	Sunrise	Sunset	Day length
4	Nuuk	Greenland	04:09:00	22:39:00	18:29:00

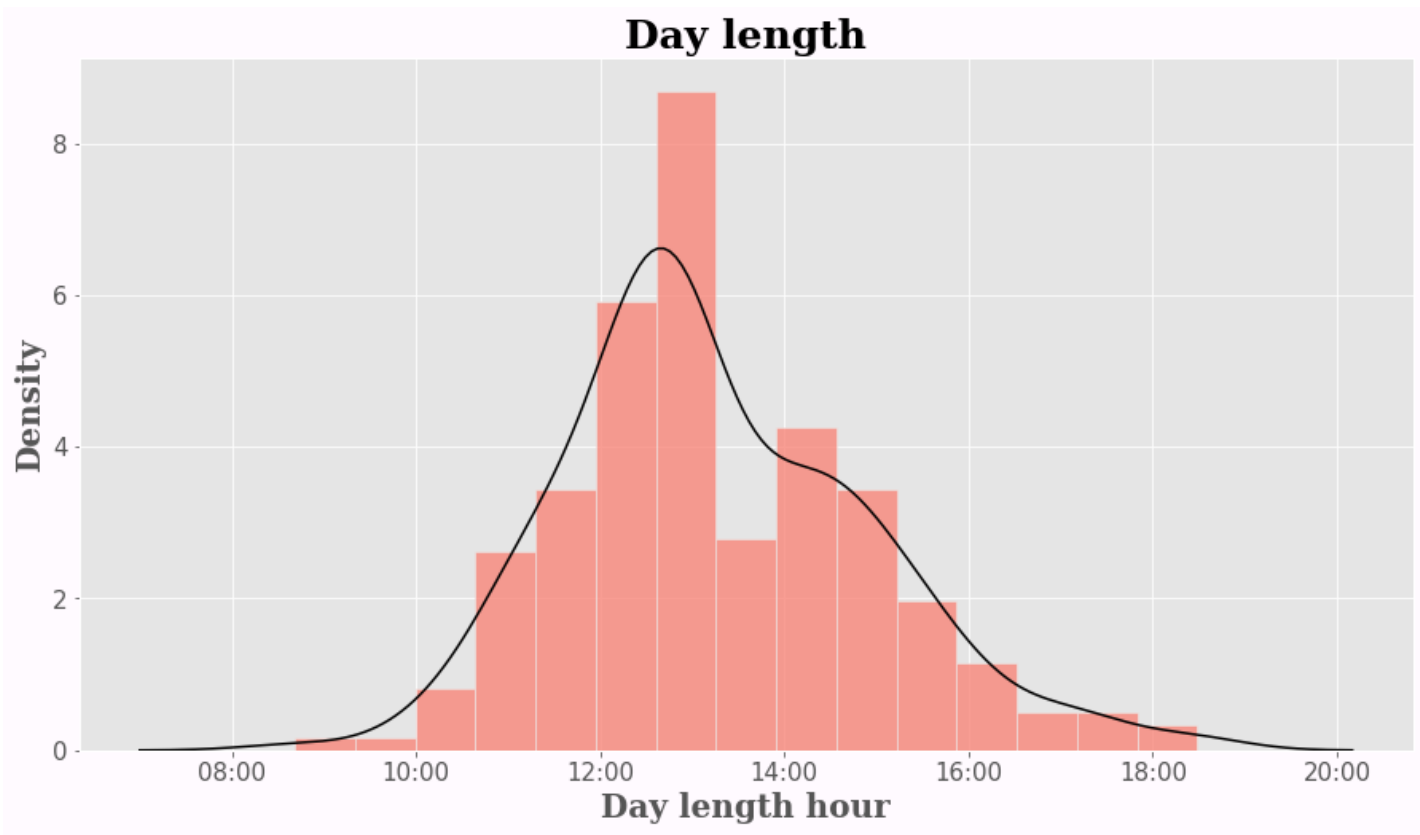
Density plots (Histogram + Kernel) for Sunrise, Sunset and Day Length

In [108…

```
def _hist(col:str):
    """
    This function plots a histogram and kernel density for a given series of data
    Input: Column Series
    Output: Histogram and KDE plot
    """
    fig,ax= plt.subplots(figsize=(15,8))
    xformatter = mdates.DateFormatter('%H:%M') #format dates for axes labels


    sns.histplot(data=df, x=col,ax=ax,color='salmon',stat='density') #Plot histogram
    sns.kdeplot(data=df, x=col,ax=ax,color='k') #Plot Kernel Density

    #Format titles, labels and background color
    ax.set_title(col, loc='center', fontsize=25, fontweight="bold",family='serif')
    ax.xaxis.set_major_formatter(xformatter)
    ax.tick_params(axis='x', labelsize=15)
    ax.tick_params(axis='y', labelsize=15)
    ax.set_xlabel(col+" hour",fontsize=20, fontweight="bold",family='serif')
    ax.set_ylabel('Density',fontsize=20, fontweight="bold",family='serif')
    fig.set_facecolor(tuple(np.array([255, 250, 255]) / 255))

_hist("Sunrise")
_hist("Sunset")
_hist("Day length")
```







The visualizations above illustrate today's sunrise and sunset times, along with the total day length. It would be insightful to compare these results with the solar data from our respective current locations.

Scatter plot and Regression Line for Sunrise and Sunset

Let's explore how sunrise and sunset timings vary with each other

In [109…

```
# Convert Day Length, Sunset, and Sunrise datetime values into total seconds.
# This numeric format is required for regression modeling (Sunset/Sunrise)
# and for generating the world map visualizations (Day Length)

def to_sec(col):
    """
    This function converts datetime values to seconds

    Input: Series with time values in the datetime format
    Output: Series with time values in seconds
    """
    return col.dt.hour * 3600 + col.dt.minute * 60 + col.dt.second

# Use the to_sec function to create new columns for Day Length, Sunset, and Sunrise in sec

df["Day l_sec"]= to_sec(df["Day length"])
df["Sunset_sec"]= to_sec(df["Sunset"])
df["Sunrise_sec"]= to_sec(df["Sunrise"])
```

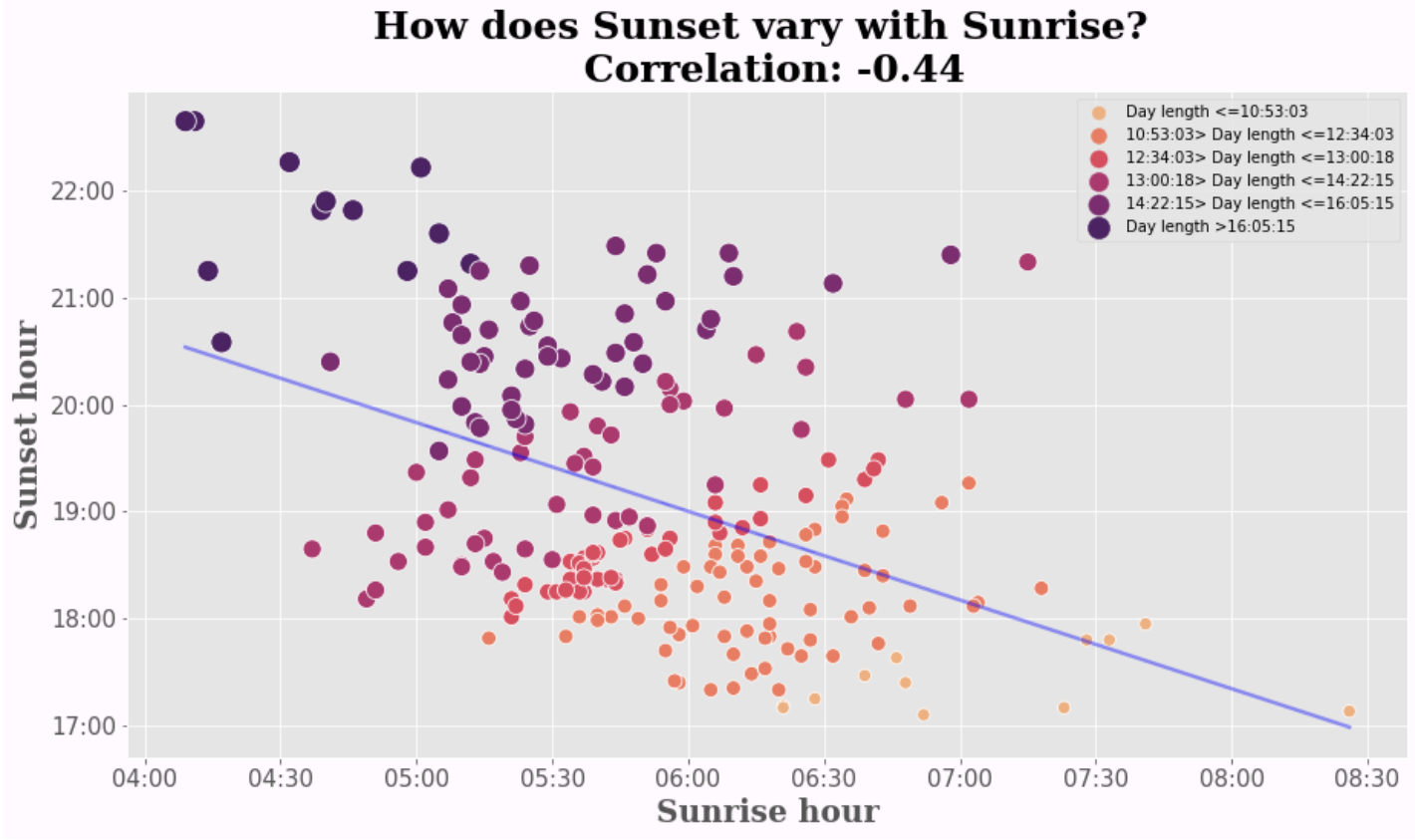
In [110…

```
# This function converts predicted regression values from seconds back into datetime forma
# This ensures that the regression line can be correctly overlaid on the original scatter
# datetime axes.

def to_dt(total_seconds):
    """
    This function convertes a value in total seconds to its respective datetime value
    Input: time in seconds
    Output: time in datetime
    """
    base_date = dt.datetime(1900, 1, 1)
    # Add the seconds to the base date
    result_time = base_date + dt.timedelta(seconds=total_seconds)
    return result_time

#Regress Sunset (in seconds) on Sunrise (in seconds) and store the preficted values in a c
#Convert the predicted seconds values to datetime using the to_dt function above

reg = smf.ols('Q("Sunset_sec") ~ Q("Sunrise_sec")',df).fit()
df['yhat'] = reg.predict()
df['yhat'] = df['yhat'].apply(to_dt)
```

In [111…

```
#Plot the scatter plot and regression

fig,ax= plt.subplots(figsize=(15,8))
xformatter = mdates.DateFormatter('%H:%M') #set format for axes labels

#Define bands depending on the magnitude of daylength according to which the points in the
#will vary in size and color

#Band 1: Bottom 5% of the Day length column values. Represented by a value of 1 in a colur
df["cols"]= np.where(df["Day length"]<=df["Day length"].quantile(0.05),1,6)
#Band 2: 5%-35%. Represented by a value of 2 in the cols column
df["cols"]= np.where((df["Day length"]>df["Day length"].quantile(0.05)) & (df["Day length"
#Band 3: 35%-55%. Represented by a value of 3 in the cols column
df["cols"]= np.where((df["Day length"]>df["Day length"].quantile(0.35)) & (df["Day length"
#Band 4: 55%-75%. Represented by a value of 4 in the cols column
```

```
df["cols"]= np.where((df["Day length"]>df["Day length"].quantile(0.55)) & (df["Day length"
#Band 5: 75%-95%. Represented by a value of 5 in the cols column
df["cols"]= np.where((df["Day length"]>df["Day length"].quantile(0.75)) & (df["Day length"
#Band 6: The remaining values are the Top 5%. They're set to the value of 6 in the cols co

#Plot scatterplot
sns.scatterplot(data=df,x="Sunrise",y="Sunset",hue="cols",size="cols",ax=ax,legend='full',


# The following function is for the legend labels
# Define a function which takes in quantile value and finds the associated Day-length valu
# corresponding to this quintile

def dayl_to_str(q):
    """
    This functions takes in a quantile values, finds the associated day length value and t
    converts it into a string value + removes extra charachters
    Input: Quantile e.g 0.05,0.35 etc
    Output: String value representine datetime
    """
    s= df["Day length"].quantile(q)
    s= str(s)[11:]
    return s

#Set new labels for the legend using the the function defined above
new_labels= ["Day length <="+dayl_to_str(0.05),
        dayl_to_str(0.05)+"> Day length <=" +dayl_to_str(0.35),
        dayl_to_str(0.35)+"> Day length <=" +dayl_to_str(0.55),
        dayl_to_str(0.55)+"> Day length <=" +dayl_to_str(0.75),
        dayl_to_str(0.75)+"> Day length <=" +dayl_to_str(0.95),
        "Day length >" +dayl_to_str(0.95)]


#Formal Legend
leg= ax.get_legend()
leg.set_title("")

leg.prop.set_size(10)
leg.prop.set_family('serif')

#Replace legend labels with new values
for t, l in zip(leg.texts, new_labels):
    t.set_text(l)

#Plot regression line
sns.lineplot(data=df,x="Sunrise",y="yhat",ax=ax, lw=2.4, color=(0,0,1,0.4),legend=False)


#Caclculate correlation to display in the title
cor= df[["Sunset_sec","Sunrise_sec"]].corr(method='pearson').iloc[0,1].round(2)
#Set title
ax.set_title("How does Sunset vary with Sunrise? \n Correlation: "+str(cor),loc='center',
#Formal axes
ax.xaxis.set_major_formatter(xformatter)
ax.yaxis.set_major_formatter(xformatter)

#Format axes ticks
ax.tick_params(axis='x', labelsize=15)
ax.tick_params(axis='y', labelsize=15)
#Format axes labels
ax.set_xlabel("Sunrise hour",fontsize=20, fontweight="bold",family='serif')
ax.set_ylabel('Sunset hour',fontsize=20, fontweight="bold",family='serif')

fig.set_facecolor(tuple(np.array([255, 250, 255]) / 255))
```



The analysis reveals a clear inverse relationship between sunrise and sunset times: later sunrises correlate with earlier sunsets and are linked with shorter day lengths.

In the visualization, cities marked in **deep purple** represent earlier sunrises and later sunsets (the longest days). Conversely, cities marked in **light orange** represent later sunrises and earlier sunsets, corresponding to the shortest day lengths in the dataset.

Plot choropleth world maps for Sunrise, Sunset and Daylengths

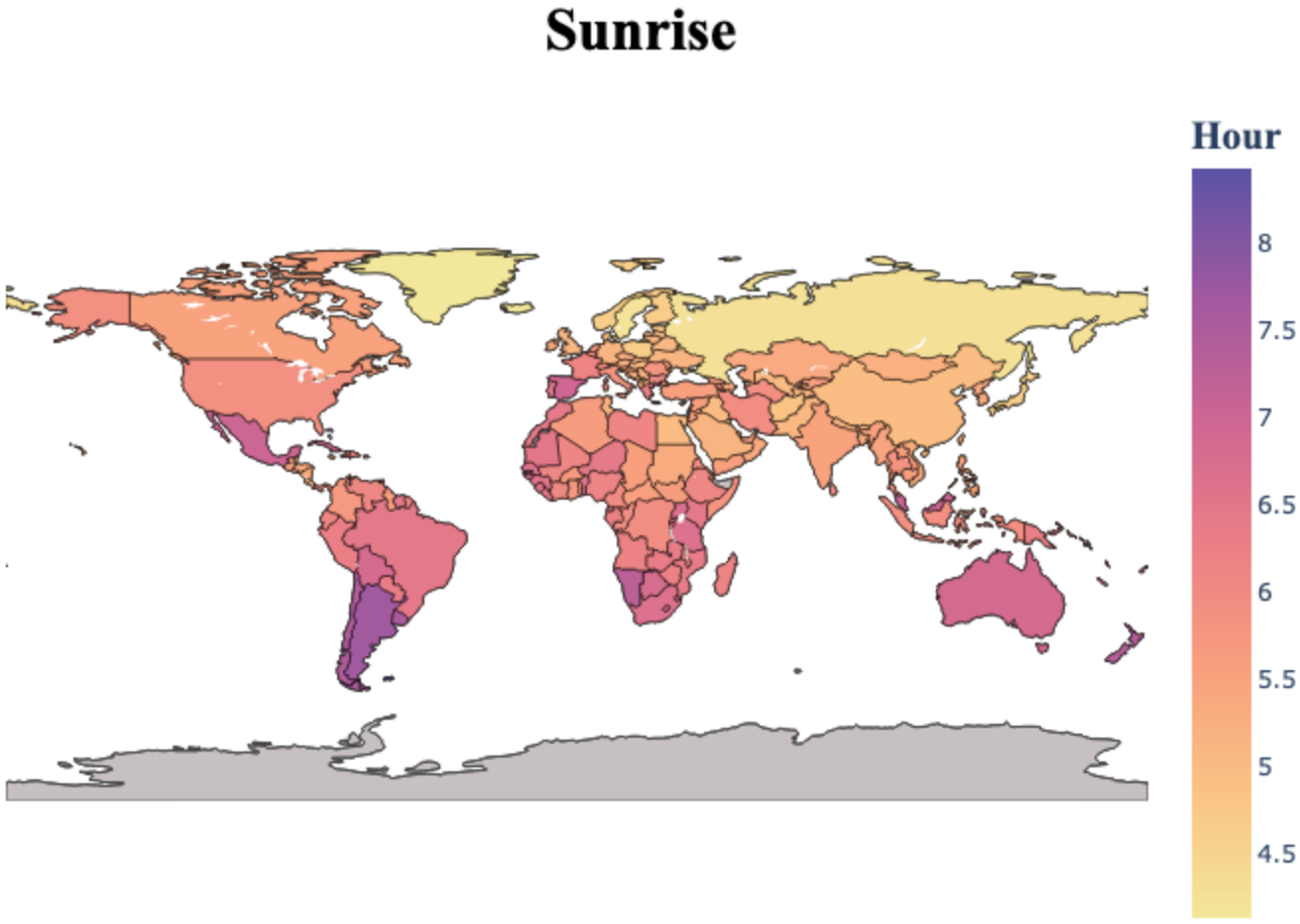
In [112…

```
#Merge iso3 country codes

ds = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/2014_world_gdp_
ds= ds.rename(columns={"COUNTRY":"Country"})

df_m= df.loc[:,["Country","Day l_sec","Sunset_sec","Sunrise_sec"]].merge(ds,on="Country",h

#Add codes for missing values
dicto={
    'South Korea':'KOR',
    'Congo-Kinshasa':'COD',
    'North Korea':'PRK',
    'Congo-Brazzaville':'COG',
    'Myanmar':'MMR',
    'Bahamas':'BHM',
    'Ivory Coast':'CIV',
    'East Timor':'TLS',
    'Reunion':"REU",
    'Curaçao':'CUR',
    'Cape Verde':'CPV',
    'Martinique':"MTQ",
    'French Guiana':"GUF",
    'Mayotte':",YT",
    'São Tomé and Príncipe':'STP',
    'Gambia':'GMB',
    'U.S. Virgin Islands':'VGB',
    'Guadeloupe':"GLP",
    'Åland':"ALA",
    'Saint Barthélemy':"BLM",
    'Micronesia':"FSM",
    'Turks and Caicos Islands':"TCA",
    'Falkland Islands':'FLK',
    'Wallis and Futuna':""}


for i in dicto.keys():
    df_m.loc[i,"CODE"]= dicto[i]

df_m= df_m.reset_index()

df_m= df_m.rename(columns={"Day l_sec":"Day length","Sunset_sec":"Sunset","Sunrise_sec":"S
```

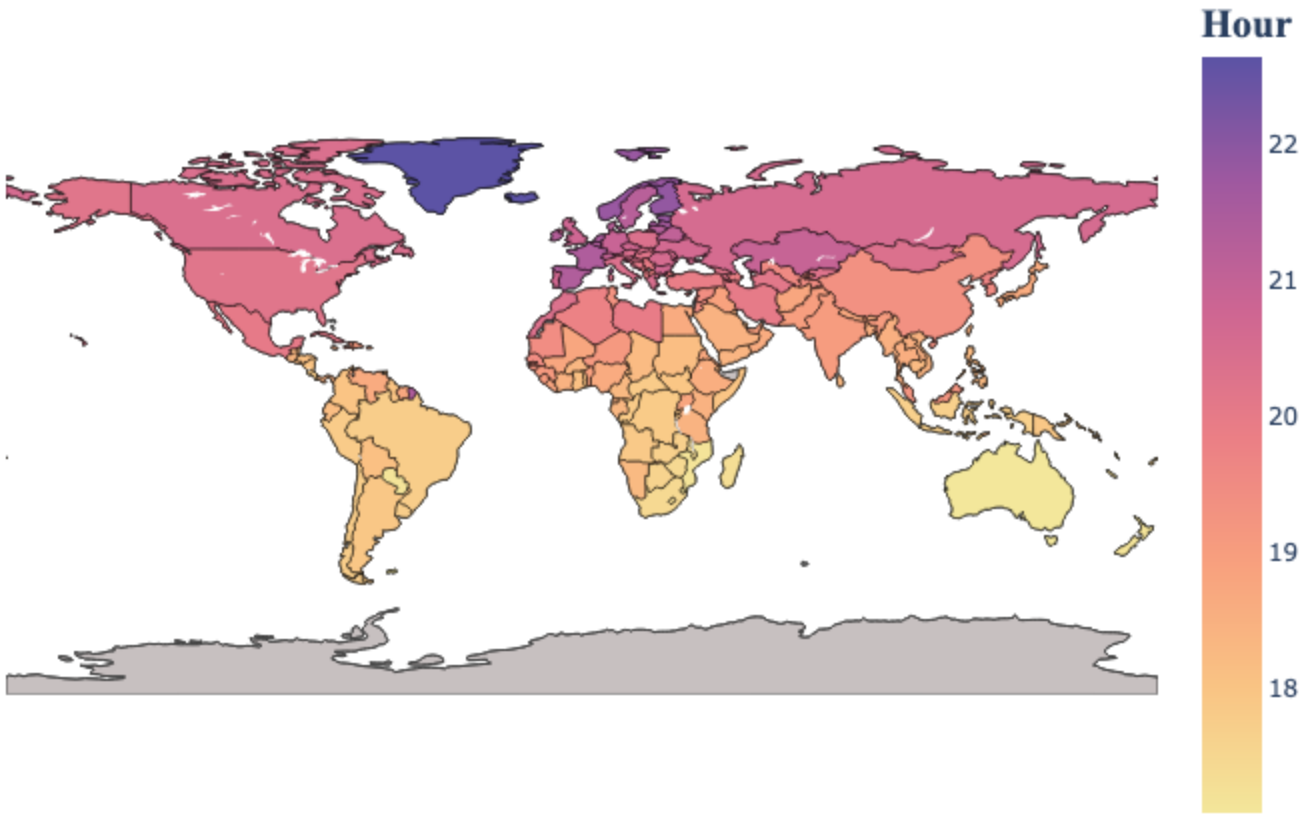
In [113…

```
#Plot maps

def map(col):
    """
    This function uses plotly to plot the maps but only outputs non-interactive images in
    Input: Relevant column whose data needs to be plotted
    Output: choropleth
    """
    fig = go.Figure(data=go.Choropleth(
        locations = df_m['CODE'], #locations identified based on country code
        z = (df_m[col]/3600).round(2),        #color varies based on magnitude of index
        text = df_m['Country'],
        colorscale = 'sunset_r',
        autocolorscale=False,
        reversescale=True,
        marker_line_color='black',
        marker_line_width=0.5,
        colorbar=dict(title={'text':'<b>Hour </b>','font':{'size':20,'family':'serif'}})
```

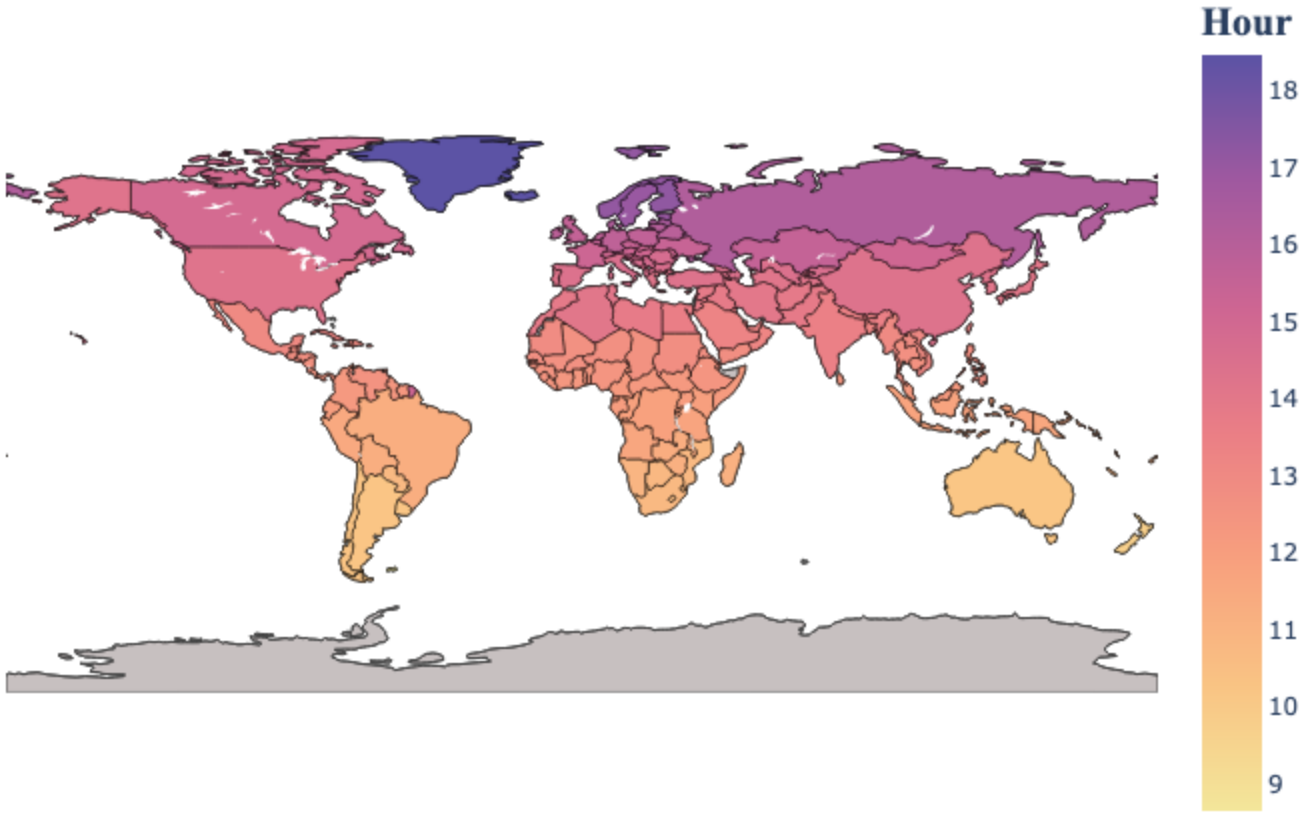
```
        ))

    fig.update_layout(
        title_text='<b>'+col,
        titlefont=dict(size =30, color='black', family='serif'),
        title_x=0.5,
        geo=dict(
        landcolor = 'rgba(175, 165, 166,0.7)', #customize grey color with missing values
        showcountries = True,
        showframe=False,
        showcoastlines=True,
        bgcolor='rgb(255,255,255)' #customize background color to our default value
        ),
        margin=dict(l=30, r=30, t=45, b=30),
        paper_bgcolor='rgb(255,255,255)',
    )

    fig.show("png")



map("Sunrise")
map("Sunset")
map("Day length")
```



Sunset



Day length



The maps above illustrate the global distribution of sunrise, sunset, and day length. How does the data for

your country compare to these global trends?

In [ ]:

```
#Convert to pdf
! jupyter nbconvert --to webpdf Sunsets_and_sunrises.ipynb
```